MP6 Report

Name: Hanbin Zheng

INTL ID: hanbin.24

ZJU ID: 3240110753

UIUC Net ID: hanbinz2

I. MP5: Memory Allocation and Running Time

1. Settings and Baseline

1. 1 Baseline 1 (tested on MacOS)

- Source Image: yaya.png (1080×1080 pixels)
- Tile Dataset: 4,731 images (75×75 pixels each)
- Mosaic Configuration: 400×400 tiles, each tile scaled to 20×20 pixels
- Output Resolution: $(400 \times 20) \times (400 \times 20) = 8000 \times 8000$ pixels

1.2 Baseline 2 (tested on Ubuntu24.04)

- Source Image: source.png (604×453 pixels)
- Tile Dataset: 4,731 images (75×75 pixels each)
- Mosaic Configuration: 400×400 tiles, each tile scaled to 5×5 pixels
- Output Resolution: $(400 \times 5) \times (400 \times 5) = 2000 \times 2000$ pixels.

2. Running Time Baseline

2.1 Baseline 1

Running time of MP5 on **MacOS**. We can see that the main bottleneck is the **Maptiles** process.

```
→  mp5 git:(public) ✗ time ./mp5 tests/yaya.png mp5_pngs 400 20
mosaic.png
Loading Source Image, costing 62.623 ms.
Loading Tile Images... (4730/4730)... 4479 unique images loaded
Loading Tile Images, costing 3563.82 ms.
Populating Mosaic: setting tile (399, 399)
Maptiles, costing 51258.7 ms.
Drawing Mosaic: resizing tiles (160000/160000)
Draw Mosic, costing: 1456.57 ms.
Saving Output Image... Save Image, costing: 3236.28 ms.
Done
./mp5 tests/yaya.png mp5_pngs 400 20 mosaic.png  53.87s user 2.53s
system 93% cpu 1:00.22 total
```

2.2 Baseline 2

Running time and memory allocation on **Ubuntu24.04**

```
(base) zhb:~/Desktop/cs225sp26/mp5$ /usr/bin/time -v ./mp5
tests/source.png ~/Desktop/mp5_pngs/ 400 5 mosaic.png
Loading Source Image, costing 39.4475 ms.
Loading Tile Images... (4730/4730)... 4479 unique images loaded
Loading Tile Images, costing 2343.6 ms.
Populating Mosaic: setting tile (399, 532)
Maptiles, costing 15324.4 ms.
Drawing Mosaic: resizing tiles (213200/213200)
```

```
Draw Mosic, costing: 145.105 ms.
Saving Output Image... Save Image, costing: 316.507 ms.
Done
  Command being timed: "./mp5 tests/source.png
/home/zhb/Desktop/mp5_pngs/ 400 5 mosaic.png"
  User time (seconds): 17.13
  System time (seconds): 1.04
  Percent of CPU this job got: 99%
  Elapsed (wall clock) time (h:mm:ss or m:ss): 0:18.20
  Average shared text size (kbytes): 0
  Average unshared data size (kbytes): 0
  Average stack size (kbytes): 0
  Average total size (kbytes): 0
  Maximum resident set size (kbytes): 1464132
  Average resident set size (kbytes): 0
  Major (requiring I/O) page faults: 0
  Minor (reclaiming a frame) page faults: 366104
  Voluntary context switches: 73
  Involuntary context switches: 126
  Swaps: 0
  File system inputs: 984
  File system outputs: 13144
  Socket messages sent: 0
  Socket messages received: 0
  Signals delivered: 0
  Page size (bytes): 4096
  Exit status: 0
```

---

3. Memory Allocation

In MP5, the memory allocation is inefficient due to the loading strategy. Below is how memory is distributed:

1. **TileImages**: In getTiles(), the program creates a `vector<TileImage>`. Each TileImage object contains a member `PNG image_` and `PNG resized_`. This means for all 4,731

tile images, the entire raw pixel data is stored on the Stack. Calculation: 4731 images $\times$ (75×75 pixels) $\times$ 32 bytes (4 $\times$ double = 32 byte) $\approx$ **850 MB** just for the source tile pool.

2. **SourceImage**: The SourceImage class stores the backingImage, but only one.
3. **MosaicCanvas**: MosaicCanvas maintains a `vector<TileImage *> myImages` of size $400 \times 400 = 160000$ pointers. Although these are just pointers to the images(8 bytes) in the tile pool, the manipulation of corresponding images is of high burden.
4. **Problems in Memory Allocation**:
   - **TileImage**: All possible tiles are loaded into memory, even if many of them are never selected by the KDTree to appear in the final mosaic. Furthermore, each TileImage keeps its original 75×75 resolution in memory even though the final output only needs a 20×20 version. From above **Baseline 2**, we can see that the Maximum **Memory Usage** is 1464132 kb $\approx$ 1.396 gb.

     ```
     class TileImage {
       private:
         PNG image_;
         PNG resized_;
         HSLAPixel averageColor_;
       /* public and private member functions.... */
     }
     ```

   - **Redundency of Copying**: An argument in function `get_match_at_idx` in `maptiles.cpp` is passed by value, instead of by reference. And this is a giant map between 4731 `Poin<3>` and `int`. This cost is totally redundant, and is the main reason of bottleneck in running time of **Maptiles**.

```
TileImage* get_match_at_idx(const KDTree<3>& tree,
                                    map<Point<3>, int>
tile_avg_map,
                                    vector<TileImage>&
theTiles,
                                    const SourceImage&
theSource, int row,
                                    int col)
```

II. MP6: Memory and Running Time

1. Running Time and Memory Allocation Improvement

Here is the results of tunning time and memory allocation of MP6, after the improvement of MP5.

1.1 Baseline 1 (MacOS)

Multi-Thread Running Time

```
→  mp6 git:(public) ✗ time ./mp6 tests/source.png ../mp5/mp5_pngs/ 400
20 mosaic.png
Loading Source Image, costing 44.6149 ms.
Loading Tile Images, costing 368.453 ms.
Maptiles, costing 33.6911 ms.
Draw Mosic, costing: 719.652 ms.
Saving Output Image... Save Image, costing: 4484.63 ms.
Done
./mp6 tests/source.png ../mp5/mp5_pngs/ 400 20 mosaic.png  7.97s user
1.21s system 158% cpu 5.784 total
```

Single-Thread Running Time

```
→  mp6 git:(public) ✗ time ./mp6 tests/source.png ../mp5/mp5_pngs/ 400
20 mosaic.png
Loading Source Image, costing 42.9117 ms.
Loading Tile Images... (4730/4730)... 4479 unique images loaded
Loading Tile Images, costing 2820 ms.
Populating Mosaic: setting tile (399, 532)
Maptiles, costing 955.312 ms.
Drawing Mosaic: resizing tiles (213200/213200)
Draw Mosic, costing: 2374.21 ms.
Saving Output Image... Save Image, costing: 4714.69 ms.
Done
./mp6 tests/source.png ../mp5/mp5_pngs/ 400 20 mosaic.png  7.45s user
2.00s system 82% cpu 11.517 total
```

1.2 Baseline 2 (Ubuntu24.04)

Multi-Threads Running Time and Memory Allocation

```
(base) zhb:~/Desktop/cs225sp26/mp6$ /usr/bin/time -v ./mp6
tests/source.png ~/Desktop/mp5_pngs/ 400 5 mosaic.png
Loading Source Image, costing 37.2346 ms.
Loading Tile Images, costing 192.582 ms.
Maptiles, costing 12.4605 ms.
Draw Mosic, costing: 119.548 ms.
Saving Output Image... Save Image, costing: 317.319 ms.
Done
  Command being timed: "./mp6 tests/source.png
/home/zhb/Desktop/mp5_pngs/ 400 5 mosaic.png"
  User time (seconds): 4.12
  System time (seconds): 0.11
  Percent of CPU this job got: 617%
  Elapsed (wall clock) time (h:mm:ss or m:ss): 0:00.68
```

```
	Average shared text size (kbytes): 0
	Average unshared data size (kbytes): 0
	Average stack size (kbytes): 0
	Average total size (kbytes): 0
	Maximum resident set size (kbytes): 260708
	Average resident set size (kbytes): 0
	Major (requiring I/O) page faults: 0
	Minor (reclaiming a frame) page faults: 69910
	Voluntary context switches: 64
	Involuntary context switches: 116
	Swaps: 0
	File system inputs: 0
	File system outputs: 13144
	Socket messages sent: 0
	Socket messages received: 0
	Signals delivered: 0
	Page size (bytes): 4096
	Exit status: 0
(base) zhb:~/Desktop/cs225sp26/mp6$
```

Single-Thread Running Time and Memory Allocation

```
(base) zhb:~/Desktop/cs225sp26/mp6$ /usr/bin/time -v ./mp6
tests/source.png ~/Desktop/mp5_pngs/ 400 5 mosaic.png
Loading Source Image, costing 32.4344 ms.
Loading Tile Images... (4730/4730)... 4479 unique images loaded
Loading Tile Images, costing 1896.58 ms.
Populating Mosaic: setting tile (399, 532)
Maptiles, costing 2295.25 ms.
Drawing Mosaic: resizing tiles (213200/213200)
Draw Mosic, costing: 792.198 ms.
Saving Output Image... Save Image, costing: 313.901 ms.
Done
	Command being timed: "./mp6 tests/source.png
/home/zhb/Desktop/mp5_pngs/ 400 5 mosaic.png"
	User time (seconds): 3.25
```

```
	System time (seconds): 1.15
	Percent of CPU this job got: 82%
	Elapsed (wall clock) time (h:mm:ss or m:ss): 0:05.33
	Average shared text size (kbytes): 0
	Average unshared data size (kbytes): 0
	Average stack size (kbytes): 0
	Average total size (kbytes): 0
	Maximum resident set size (kbytes): 251352
	Average resident set size (kbytes): 0
	Major (requiring I/O) page faults: 0
	Minor (reclaiming a frame) page faults: 67579
	Voluntary context switches: 136
	Involuntary context switches: 46
	Swaps: 0
	File system inputs: 8
	File system outputs: 13144
	Socket messages sent: 0
	Socket messages received: 0
	Signals delivered: 0
	Page size (bytes): 4096
	Exit status: 0
```

---

2. Memory Allocation Analysis

The memory management in MP6 undergoes a fundamental paradigm shift from "pre-loading everything" to "**Lazy Loading** and **Lightweight Metadata**." In MP6, the `TileImage` class no longer holds a `PNG` object as a member. Instead, it only stores the `HSLAPixel averageColor_` and the a `std::string filename_` . This reduces the memory footprint of the tile images,and it allows the application to handle significantly larger tile datasets without exhausting system RAM. Pixel data (original PNG) is only allocated when necessary—during the final `drawMosaic` phase.

Inside drawMosaic, I implemented a `std::unordered_map<TileImage*, PNG>` to cache resized images. Since a PhotoMosaic often reuses the same tile multiple times, this ensures that each unique image is loaded and resized exactly once.

Data Flow:

1. **getTiles**: Reads file list, and stores only `averageColor_` and `filename_` (Low Memory)
2. **mapTiles**: Performs KDTree search using metadata only.
3. **drawMosaic**: For each tile, if not in cache $\rightarrow$ Load from disk $\rightarrow$ Resize $\rightarrow$ Store in cache $\rightarrow$ Paste. (Efficient Memory Usage)

---

3. Running Time Analysis

The optimized MP6 implementation shows a drastic runtime performance improvement, reducing the total time from around **60s** to approximately **5.784s** (on MacOS) and from **18.2s** to **0.68s** (on Ubuntu24.04). This is done by following methods:

1. **Maptiles Datastructure/Memory Optimization**: Time dropped from **15324.4 ms** to **12.4605 ms**. This giant speedup, primarily due to eliminating redundant copying and switching from $O(\log(n))$ `std::map` tree searching to $O(1)$ `std::unordered_map` hash table lookups.
2. **Parallel Processing**: By utilizing **std::thread** for parallel processing, the time spent on I/O and pixel manipulation (**getTile**, **MapTile** and **drawMosaic**) is scaled with respect to the number of CPU cores.

III. Changes to Reduce Memory Footprint and Running Time

> To achieve the performance goals of MP6, the following five optimizations were implemented:

1. Pass-by-Reference and Hash Map Optimization

In maptiles.cpp, the original code passed large map objects by value, triggering massive copy overhead for every tile. In MP6, I hanged this parameters to const reference to eliminate copying. Besides, MP5 use `std::map` to match `Point<3>` and `int`, which is underlying a tree, with searching complexity $O(\log(n))$. I change this implementation to `std::unordered_map`, a hash table with average searching complexity $O(1)$. The change from passing value to passing reference largely reduced the running time, and the shift from `std::map` to `std::unordered_map` also provided a 30% speedup in the mapping phase.

2. Memory Footprint Reduction (Metadata-Only Loading)

I redesigned `TileImage`, `mapTiles`, and `MosaicCanvas` to prevent loading thousands of PNGs into memory at startup. By only storing the filename and the average color, the system's peak memory usage is significantly lowered (On **Ubuntu24.04**, from around **1.396gb** to around **245.461 mb**). Full image data is only fetched from disk during the final rendering (`drawMosaic`), and a local cache ensures no redundant I/O for duplicate tiles.

3. Multi-threaded Parallel Processing

I Implemented multi-threading using `std::thread` :

1. **Loading Tiles**: Parallelizing disk I/O and color calculation.
2. **Mapping Tiles**: Distributing KDTree searches across multiple cores.
3. **Drawing Mosaic**: Parallelizing the resizing and pasting of tiles.

This optimization significantly increased CPU utilization and reduced the total running time, making the generation of 2000×2000 mosaics nearly instantaneous.

4. Optimized the KDTree's nearest neighbor search by:

Removing unnecessary square root operations (comparing squared distances).
Implementing a specialized 3D distance function to reduce function call overhead.

Using conditional compilation (`#if`) to tailor the optimization for different CPU architectures (Apple Silicon M3 vs. Intel i9), ensuring maximum performance across platforms.

5. Cache-Friendly SourceImage Pre-calculation

In the original implementation, `getRegionColor` recalculated the average color of a region every time it was called. I modified the `SourceImage` constructor to pre-calculate all region colors and store them in a `std::vector<HSLAPixel>` . This way, everytime `getRegionColor` is called, we simply fetch an element from a vector with index, which is $O(1)$. While this didn't reduce the total number of calculation, this makes the operation cache-friendly. Howeveer, this change doesn't achieve a massive performance boost in testing (actually, it is negligible).